

Mandatory exercise set I

Cryptographic attacks and secure communication

Release: 22/09/2026 **Deadline:** 06/10/2026

Rosario has finished marking the security exam. SAP staff need to send grades to the grading system. In this assignment, you investigate several deliberately insecure versions of this system. The examples use fictional accounts and student identifiers. The systems in different exercises are separate versions, so do not assume they share keys or encryption formats.

Complete all four exercises individually. You may use a programming language of your choice.

Provided files

`BAD_cipher.py` and `cipher_challenge.json` specify Exercise 3. `grading_lab.pcap` is the capture for Exercise 4.

Hand-in

Upload the following two files to LearnIT:

1. A PDF report of max 4-6 pages. Organise it by exercise. Explain your approach, present the main results, and interpret what they show.
2. A ZIP archive containing your source code, a README with exact setup and run commands, the recovered file from Exercise 4.

Implement your own attack and search logic. Your report must explain the code you submit.

Exercise 1: The ElGamal job

SAP transmits a student identifier using textbook ElGamal encryption. The public parameters are

$$p = 29837, \quad g = 42, \quad PK = g^x \bmod p = 22690.$$

For this exercise, messages (i.e. student identifiers) are integers in $\{1, \dots, p-1\}$. An encryption of a message m has the form

$$(c_1, c_2) = (g^r \bmod p, m PK^r \bmod p),$$

where r is the encryption randomness. You intercept

$$c_1 = 23447, \quad c_2 = 8372.$$

- 1a. Recover the private key and plaintext.** Write code that finds an exponent x satisfying the public-key equation and decrypts the intercepted ciphertext. State the search method, the recovered exponent, and the student identifier. Explain how you check your result.
- 1b. Modify the ciphertext without the private key.** Assume that you know the intercepted message's plaintext from observing SAP enter it. The fictional target identifier is $m' = 12345$. Write a separate function that changes the ciphertext so that it decrypts to 12345.
Note: The attack function may use only p , the public parameters, the intercepted ciphertext, the known plaintext, and the target identifier. It must not use the private key, recover a discrete logarithm, or use the encryption randomness. Keep c_1 unchanged and modify only c_2 . Do not generate a new encryption. Give the modified ciphertext and justify your construction algebraically. You may use the recovered private key afterwards to verify the result.
- 1c. (optional) Interpret the attacks.** Which attack would become computationally harder with appropriately chosen, sufficiently large parameters? Which attack would still work? Explain which security property each attack violates and why increasing the key size alone does not address both problems.

Evidence to include

Give the recovered key and plaintext, the modified ciphertext, the verification output, and the reasoning behind both attacks. The report should make clear that the ciphertext-modification algorithm does not depend on private-key recovery.

Exercise 2: Hash me if you can

For a UTF-8 encoded message m , define

$$H_b(m) = \text{the first } b \text{ bits of SHA-256}(m), \quad b \in \{16, 24, 32\}.$$

The first bits are the leftmost bits in the conventional hexadecimal digest. For these values of b , retain the first 2, 3, or 4 digest bytes, respectively. You may use a standard SHA-256 implementation.

- 2a. Implement collision search.** For each output length, find distinct messages $m_1 \neq m_2$ such that $H_b(m_1) = H_b(m_2)$.
- 2b. Distinguish collision and second-preimage resistance.** How does the task change if m_1 is fixed in advance and you must find a different m_2 with the same hash? Compare the expected work for generic attacks in the two settings. You do not need to implement a second-preimage search. Does your experiment show that full SHA-256 is practically broken? Explain your answer.

Exercise 3: Breaking BAD

A developer proposes BAD, a block cipher with 128-bit blocks and two independent 128-bit keys, K_1 and K_2 . For one plaintext block P , encryption is

$$E_{K_1, K_2}(P) = R(P \oplus K_1) \oplus K_2,$$

where $\oplus$ is bitwise XOR and R rotates the entire 128-bit block left by 13 bits. A block is interpreted as an unsigned big-endian integer. Bits shifted out of the left side wrap around to the right. There is no padding and no multi-block mode of operation.

`BAD_cipher.py` contains the public encryption algorithm and a public test vector. The test-vector keys are unrelated to the challenge keys. `cipher_challenge.json` contains one known plaintext/ciphertext pair, three challenge ciphertext blocks, and a target plaintext. All these ciphertexts use the same unknown keys. The hexadecimal strings describe the exact bytes; trailing space bytes are part of a plaintext and must be preserved.

- 3a. Derive decryption.** Give a formula for decryption when both keys are known, and explain why it inverts encryption.
- 3b. Recover the challenge plaintexts.** Find an attack using the known plaintext/ciphertext pair. Write code that recovers all three challenge plaintexts without exhaustive key search. Report the recovered bytes in hexadecimal and as readable ASCII. Explain why the attack works for any plaintext block encrypted under these same keys.
- 3c. (optional) Assess the design.** Does your attack recover the original pair (K_1, K_2) ? Is recovering that exact pair necessary? Explain why the nominal 256-bit key length does not establish security. Describe what happens to the ciphertext difference when two plaintexts differ in exactly one bit. Would repeatedly applying rounds of this same form with additional independent keys remove the underlying weakness?

Evidence to include

Provide the decryption formula, the reasoning behind your attack, all recovered plaintexts, and the target ciphertext.

Exercise 4: The grades are in the air

Open `grading_lab.pcap` in Wireshark. It is a purpose-built teaching capture of a fictional grading network, containing several short connections and some background traffic. Network packets are synthesised, and the TLS records come from a real TLS 1.3 exchange. All accounts and identifiers are fictional. **Analyse the supplied file offline. You do not need to contact its hosts.**

Support each finding with packet numbers or TCP stream identifiers, relevant field values, and the display filter or reconstruction method you used. Packet numbers mean frame numbers in the original, unedited capture.

- 4a. Map the network.** Identify the IPv4 hosts and their likely roles. List the relevant application and transport protocols and server ports. Recover the queried domain names and their returned addresses. Distinguish roles observed directly from roles you infer.
- 4b. Recover exposed application data.** Recover the login username and password. Identify the session value returned by the server and show how it is used in later requests. Recover the student identifiers and grades, and describe the corresponding server responses.
- 4c. Recover the transferred file.** Export the downloaded grades file using Wireshark, or reconstruct its HTTP response body. Submit the recovered file, its exact byte length, and its full SHA-256 digest.
- 4d. (optional) Reconstruct the timeline.** Give a short sequence of the login, grade submissions, and file download. Compare the source hosts of the grade submissions and the session values they use. Identify any suspicious behaviour. Explain what the capture establishes and what it does not establish about the people or permissions involved.

Use stream reassembly when an application message spans several packets. Useful Wireshark features include Protocol Hierarchy, Conversations, Follow TCP Stream, and Export Objects. Screenshots help illustrate your findings.